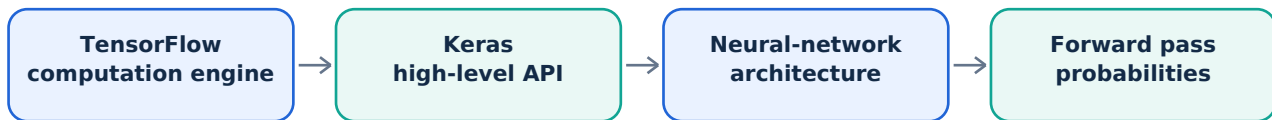


TensorFlow & Keras Foundations

Tensors, Sequential models, Dense layers, parameter counting, and untrained forward passes



Day 2 outcome

You moved from manually programming one NumPy neuron on Day 1 to defining complete neural-network architectures with Keras. You inspected tensor properties, calculated trainable parameters, examined layer structures, and ran predictions through untrained models.

Environment verified

Component	Installed version	Role
Python	3.12.10	Programming environment
TensorFlow	2.21.0	Tensor operations and execution engine
Keras	3.15.1	High-level neural-network API

Core learning objectives

- Explain how TensorFlow and Keras work together.
- Read tensor rank, shape, and datatype correctly.
- Separate the batch dimension from the feature dimension.
- Build Sequential networks using Input and Dense layers.
- Calculate weights, biases, and total parameters manually.
- Interpret model summaries and untrained forward-pass outputs.

Status: Week 4 Day 2 completed

1. Tensor fundamentals

A tensor is a multidimensional collection of values with one consistent datatype. Tensors are similar to NumPy arrays, but standard TensorFlow tensors are immutable: operations create new tensors instead of changing the original tensor in place.

Example	Rank	Shape	Meaning
7.0	0	()	One scalar value
[0.20, 0.80, 0.70]	1	(3,)	One vector with three values
Four customers x three features	2	(4, 3)	Four rows and three columns

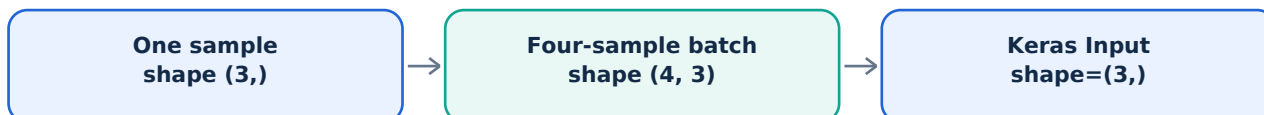
Rank, shape, and datatype

- Rank is the number of axes or dimensions.
- Shape reports the size along each axis.
- Datatype reports how values are stored, such as float32.

Your customer tensor

```
customer_tensor = tf.constant([
    [0.10, 0.90, 0.80],
    [0.85, 0.40, 0.10],
    [0.40, 0.60, 0.50],
    [0.70, 0.75, 0.20]
], dtype=tf.float32)
```

Sample shape versus batch shape



Why Input(shape=(3,))?

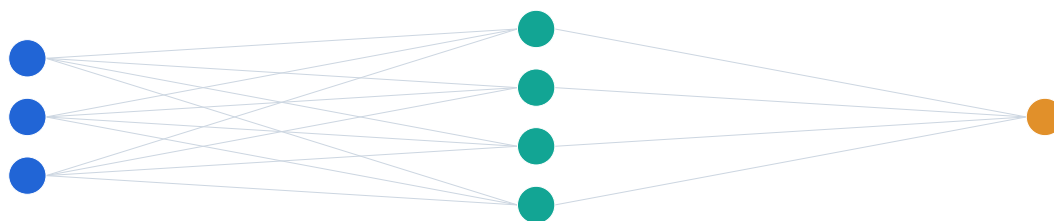
Keras describes the feature shape of one sample. The number of customers belongs to the batch dimension and can vary. The same model can accept 1, 4, 32, or 100 customers, but every customer must still supply exactly three features.

In an output shape such as (None, 6), None means the batch size is variable. The second dimension remains fixed at six neuron outputs per sample.

2. Keras architecture building blocks

A Sequential model passes one tensor through each layer in order. It is suitable for a simple stack in which every layer has one input and one output.

First model: 3 inputs -> 4 ReLU neurons -> 1 sigmoid neuron



3 inputs

4 ReLU neurons

1 sigmoid output

`keras.Input(shape=(3,))` defines the expected feature shape. It is not a trainable Dense layer, does not perform a weighted calculation, and does not appear in `model.layers`.

Dense layer calculation

Every Dense neuron receives every output from the previous layer. Its operation is $\text{activation}(XW + b)$. Keras calls W the kernel. A Dense layer therefore contains a kernel matrix, a bias vector, and an activation function.

Parameter formula

Parameters = (number of inputs x number of neurons) + number of neuron biases

Layer	Kernel	Bias	Total
Hidden: 3 -> 4	$3 \times 4 = 12$	4	16
Output: 4 -> 1	$4 \times 1 = 4$	1	5
Complete model	$16 + 5$	-	21

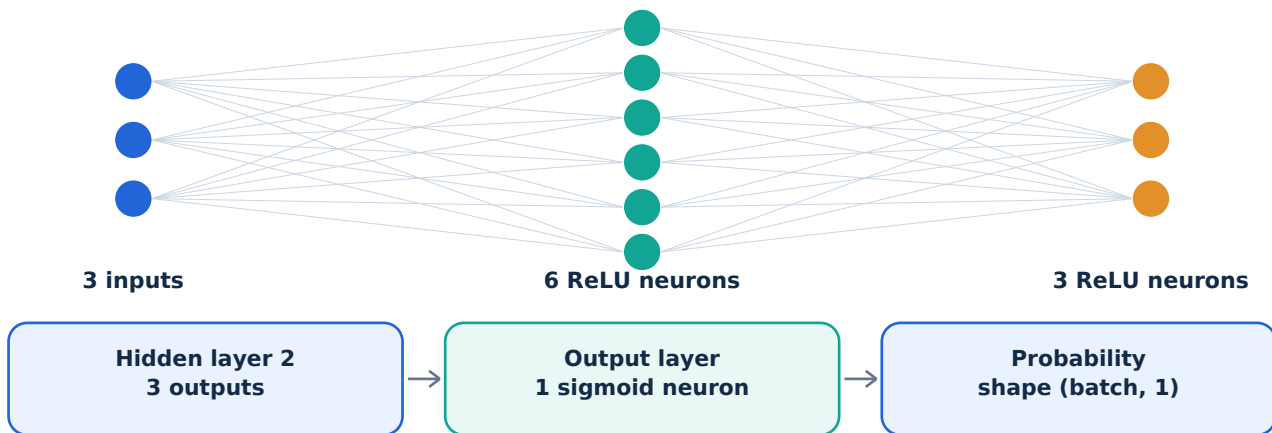
Model summary interpretation

Layer	Output shape	Parameters
hidden layer	(None, 4)	16
output layer	(None, 1)	5

The hidden layer kernel shape is (3, 4), and its bias shape is (4,). The output layer kernel shape is (4, 1), and its bias shape is (1,).

3. Mini-project: Keras Architecture Inspector

Model name: deep_churn_network



Manual parameter calculation

Layer	Kernel weights	Biases	Layer total
hidden_layer_1: 3 -> 6	$3 \times 6 = 18$	6	24
hidden_layer_2: 6 -> 3	$6 \times 3 = 18$	3	21
output_layer: 3 -> 1	$3 \times 1 = 3$	1	4
Network total			$24 + 21 + 4 = 49$

Layer inspection

Layer	Kernel shape	Bias shape	Activation
hidden_layer_1	(3, 6)	(6,)	ReLU
hidden_layer_2	(6, 3)	(3,)	ReLU
output_layer	(3, 1)	(1,)	sigmoid

Why one output neuron?

Binary churn classification needs one score. Sigmoid converts the output neuron's raw value into a number between 0 and 1. A threshold then maps that probability-like score to class 0 or class 1.

All 49 parameters are trainable. No parameters were actually trained on Day 2 because `model.fit()` was intentionally not used.

4. Forward pass and classification thresholds



A direct model call performs forward propagation. It does not calculate loss, backpropagate gradients, or update parameters. Therefore, it does not train the model.

Observed mini-project run

Customer	Probability	Class at 0.50	Class at 0.65
1	0.5000	1	0
2	0.5217	1	0
3	0.5000	1	0
4	0.4988	0	0

Result: the probability tensors were identical, while three class labels changed. A value of exactly 0.50 satisfies probability ≥ 0.50 , but it does not satisfy probability ≥ 0.65 .

Why results can differ between runs

Before training, Keras initializes kernels using random values and biases commonly begin at zero. A new run may therefore produce different outputs. Values of exactly 0.50 can also occur in an untrained ReLU network when the final pre-sigmoid value is zero. This is not an error.

Probability verification

```
probabilities_50 = deep_churn_network(customer_tensor, training=False)
classes_50 = tf.cast(probabilities_50 >= 0.50, tf.int32)

probabilities_65 = deep_churn_network(customer_tensor, training=False)
classes_65 = tf.cast(probabilities_65 >= 0.65, tf.int32)

same = tf.reduce_all(tf.equal(probabilities_50, probabilities_65))
```

Important: probabilities from this untrained model are valid mathematical outputs, but they are not meaningful business predictions because the network has not learned from labeled churn data.

5. Build, compile, train, and predict

Operation	Purpose	Does it train?
Build model	Defines layers and architecture	No
model.summary()	Displays shapes and parameter counts	No
model.compile()	Selects optimizer, loss, and metrics	No
model.fit()	Runs training and updates parameters	Yes
model.evaluate()	Measures performance	No
Direct call / predict()	Performs a forward pass	No

Common mistakes corrected today

- A Python variable name does not set the Keras model name. Pass name='deep_churn_network' to Sequential.
- probabilities == probabilities is a self-comparison. Compare the two forward-pass tensors instead.
- A threshold may change class labels, never the underlying model outputs.
- The Input object is not a hidden layer and does not appear in model.layers.
- If the model expects shape (batch, 3) and receives shape (4, 2), Keras raises a shape incompatibility or dimension-mismatch error.

Five-minute revision

Question	Quick answer
TensorFlow vs. Keras?	TensorFlow is the computation framework; Keras is the high-level neural-network API.
Rank of shape (4, 3)?	Rank 2: four samples, three features per sample.
Meaning of Dense(6)?	Six neurons and six output values per sample.
Dense parameter formula?	inputs x neurons + neuron biases.
Meaning of None?	Variable batch size.
Does compile train?	No. model.fit() begins training.
Day 1 connection?	Dense automates the same activation($XW + b$) calculation.

Day 2 summary

You can now construct and inspect an untrained Keras network, reason about tensor dimensions, calculate every Dense-layer parameter, and distinguish raw model outputs from threshold-based classes. Day 3 will connect these foundations to loss functions, backpropagation, optimizers, epochs, and actual training.

Week 4 progress: 2/7 complete